Reutilização no MultiPag: GRU e DRU

Disciplina: Desenvolvimento Orientado a Reuso (PUCPR)

Equipe: Eduardo Lourenço da Silva, Gabriel Pereira Lugon Moulin e João Pedro Novak Rausis

1. Sobre a avaliação

Verde: de acordo com o resultado esperado. **Amarelo**: identificação parcial. **Vermelho**: não encontrado.

O guia traz 5 resultados para o GRU e 9 para o DRU. Classificamos todos e detalhamos a adequação de DRU 6 a 9, como pede o enunciado.

Resultado: no GRU ficamos com 1 verde, 3 amarelos e 1 vermelho. No DRU, com 1 verde, 7 amarelos e 1 vermelho.

1.1 Ativos reutilizáveis do MultiPag

Ativos considerados nas duas avaliações:

Ativo	Tipo	Onde está	Quem reutiliza
multipag-validador-documentos	Pacote npm (v1.0.0)	pacotes/validador-documentos e npm	Cpf, Cnpj e formato.ts (telas)
GatewayPagamento e Adapters (Pix, Cartão, Cripto)	Interface e classes adaptadoras	src/gateways	Processadores de pagamento e rota de webhooks
ProcessadorPagamentoTemplateMethod	Classe base (Template Method)	src/pagamentos	Processadores de Pix, Cartão, Boleto e Cripto
CrudServiceTemplateMethod	Classe base (Template Method)	src/crud	Serviços de Business, Cobrança e Saque
ExportadorExtratoTemplateMethod	Classe base (Template Method)	src/extrato	Exportadores CSV e TXT e a tela de extrato
AppConfigSingleton e DatabaseSingleton	Classes Singleton	src/config e src/infra	Processadores, Adapters e repositórios
Componentes e hooks das telas	Componentes React	app/_componentes e app/_lib	Todas as telas
Modelo de características	Modelo de domínio (FeatureIDE)	docs/featureide/MultiPag	Documentação do projeto

2. GRU: Gerência de Reutilização

Resultado	O que o guia pede	Cor	Justificativa
GRU 1	Estratégia de gestão de ativos documentada (definição de ativo e critérios de aceitação, certificação, classificação, descontinuidade e avaliação).	**Amarelo**	O README identifica os pontos de reuso e explica por que cada um faz sentido. Mas não há documento de estratégia nem critérios definidos.
GRU 2	Mecanismo de armazenamento e recuperação de ativos implantado.	**Verde**	O pacote multipag-validador-documentos está publicado no npm (v1.0.0) e é recuperado com npm install, como o MultiPag já faz. O código-fonte está no GitHub, com README e licença MIT. Os outros ativos ficam só no repositório.

Resultado	O que o guia pede	Cor	Justificativa
GRU 3	Dados de utilização dos ativos registrados.	**Amarelo**	A tabela "Quem usa" do README e o package-lock.json registram quem usa o pacote e em qual versão. É feito à mão e só para o pacote, sem datas nem problemas encontrados.
GRU 4	Ativos mantidos periodicamente, com modificações controladas.	**Amarelo**	O pacote tem versão semântica, 11 testes e histórico no Git. Falta critério de manutenção, CHANGELOG, tags de versão e integração contínua.
GRU 5	Usuários notificados sobre problemas, modificações, novas versões e descontinuação.	**Vermelho**	Não há CHANGELOG, releases no GitHub, aviso de versão descontinuada nem canal para reportar problemas.

3. Como adequar o protótipo ao GRU

Os documentos novos ficariam na pasta docs/reuso do repositório.

Resultado	O que fazer
GRU 1	Escrever docs/reuso/estrategia-de-reuso.md com: o que é ativo reutilizável (tem mais de um uso, README, testes e versão); critérios de aceitação (testes passando, README com exemplo, sem dependência do MultiPag); certificação (revisão de outro membro com checklist); classificação (por tipo e por domínio); descontinuidade e avaliação a cada sprint.
GRU 2	Já está de acordo. Para melhorar, criar docs/reuso/catalogo.md listando todos os ativos (nome, tipo, local, versão e como usar), incluindo Adapters e classes base.
GRU 3	Acrescentar ao catálogo as colunas "usado em", "versão" e "data", e anotar a cada sprint os downloads do pacote no npm e os problemas encontrados.
GRU 4	Criar o CHANGELOG.md do pacote, marcar versões com tags no Git, rodar npm test em cada pull request (GitHub Actions) e fazer uma revisão de manutenção por sprint.
GRU 5	Publicar um release no GitHub a cada versão, usar npm deprecate nas versões descontinuadas e avisar por issue quem usa o ativo (lista no catálogo).

4. DRU: Desenvolvimento para Reutilização

Resultado	O que o guia pede	Cor	Justificativa
DRU 1	Domínios de aplicação identificados, com o potencial de reuso.	**Verde**	O domínio é cobrança e pagamentos digitais. O README aponta os pontos de reuso: validação de documentos (pacote), Adapters, Template Methods e componentes de tela.
DRU 2	Capacidade de reutilização avaliada, com ações corretivas.	**Amarelo**	Houve uma ação corretiva (a regra de CPF e CNPJ estava duplicada e virou pacote), mas não houve avaliação da capacidade de reuso nem medidas.
DRU 3	Programa de reutilização planejado (propósitos, escopo, metas e objetivos).	**Amarelo**	O README traz o propósito e o escopo do reuso. Faltam metas, objetivos mensuráveis, responsáveis e prazos.
DRU 4	Programa de reutilização implantado, monitorado e avaliado.	**Vermelho**	Não há programa implantado, nem indicadores ou acompanhamento do uso dos ativos.
DRU 5	Propostas de reutilização avaliadas para a aplicação alvo.	**Amarelo**	Os reusos foram justificados no README e conferidos por testes. Falta um checklist para avaliar uma proposta antes de adotá-la.
DRU 6	Formas de representação escolhidas para modelo e arquitetura de domínio.	**Amarelo**	Usamos o modelo de características (FeatureIDE), casos de uso e diagrama do banco. A arquitetura de domínio só aparece como estrutura de pastas no README.

Resultado	O que o guia pede	Cor	Justificativa
DRU 7	Modelo de domínio com limites e relações com outros domínios, capturando comum, variante, opcional e obrigatório.	**Amarelo**	O modelo de características captura obrigatórias, opcionais e variantes (grupos Or e Alternativos). Faltam os limites do domínio e as relações com outros domínios (Pix/DICT, adquirente de cartão, corretora de cripto, Receita Federal).
DRU 8	Arquitetura de domínio descrevendo uma família de aplicações, mantida no ciclo de vida.	**Amarelo**	O código tem pontos de variação (GatewayPagamento com Adapters, classes filhas dos Template Methods, configuração por ambiente), mas nenhum documento descreve a família de aplicações.
DRU 9	Ativos do domínio especificados, desenvolvidos e mantidos.	**Amarelo**	O pacote validador está desenvolvido, publicado e documentado. Os outros ativos não têm especificação própria e não há plano de manutenção.

5. Como adequar o protótipo ao DRU

5.1 DRU 6, 7, 8 e 9

Resultado	O que fazer
DRU 6	Registrar em docs/reuso/representacoes.md o que foi escolhido e por quê: modelo de características no FeatureIDE (já existe) para o modelo de domínio e um diagrama de componentes UML para a arquitetura.
DRU 7	Completar o modelo com um diagrama de contexto (limite do domínio e relação com Pix/DICT, adquirente de cartão, corretora de cripto e Receita Federal) e atualizar o model.xml sempre que um método de pagamento, formato ou tipo de conta mudar.
DRU 8	Escrever docs/reuso/arquitetura-de-dominio.md com o núcleo comum (domain, repositorios, services, http), os pontos de variação ligados ao modelo de características e como derivar produtos (por exemplo, só Pix e cartão ou completo) por configuração.
DRU 9	Fazer uma ficha por ativo no catálogo (interface, entradas e saídas, exemplo, testes e versão), extrair GatewayPagamento e os Adapters para um pacote npm e manter com versão, testes e CHANGELOG, como no GRU 4.

5.2 DRU 1 a 5

Resumo para os cinco primeiros resultados.

Resultado	O que fazer
DRU 1	Já está de acordo. Manter a lista de domínios na estratégia de reuso.
DRU 2	Checklist por sprint com o percentual de código reaproveitado e as regras duplicadas encontradas.
DRU 3	Plano de reuso com propósito, escopo, metas, responsáveis e prazos.
DRU 4	Executar o plano e acompanhar por indicadores (ativos, usos e defeitos abertos) a cada sprint.
DRU 5	Checklist antes de adotar um ativo (compatibilidade, custo de adaptação, risco e testes).